Variational Auto-encoder

Heqing Shi

FinTech @ University of Edinburgh Business School
Heqing.Shi@ed.ac.uk
http://shiheqing.github.io

October 31, 2025

Keywords: Generative model, Latent space, Dimensionality reduction, ELBO

1 Motivation

PCA, or Auto-encoder, projects high-dimensional data to lower dimensional. But in many more cases, we are interested in the underlying distribution $Pr(\mathbf{x})$ that governs the data. Unfortunately, it is usually prohibitive to learn $Pr(\mathbf{x})$ from $\mathbf{x}$ in a data-driven way, and, at the same time, parametric assumptions are too restrictive.

Finding a way to learn $Pr(\mathbf{x})$ non-parametrically is of paramount meaning. This allows us to sample "realizations" from $Pr(\mathbf{x})$ that are indistinguishable from $\mathbf{x}$. We will show that how **Variational Auto-encoder** tackles this, by using the idea of Auto-encoder and a very tractable prior distribution (usually standard Gaussian).

2 Latent Variable Model

In order to describe a probability distribution $Pr(\mathbf{x})$ over a multi-dimensional variable $\mathbf{x}$, a latent variable model choose to model a joint distribution:

$$Pr(\mathbf{x}, \mathbf{z}) \tag{1}$$

of the data $\mathbf{x}$, and an unobserved (hence *latent*) variable $\mathbf{z}$. Using the marginalization trick, we have:

$$Pr(\mathbf{x}) = \int_{\mathbb{R}} Pr(\mathbf{x}, \mathbf{z}) d\mathbf{z}. \tag{2}$$

By applying the Baye's rule, the joint distribution $Pr(\mathbf{x}, \mathbf{z})$ can be factorized as:

$$Pr(\mathbf{x}, \mathbf{z}) = Pr(\mathbf{x}|\mathbf{z}) Pr(\mathbf{z}). \tag{3}$$

The above allows (2) to be written as:

$$Pr(\mathbf{x}) = \int_{\mathbb{R}} Pr(\mathbf{x}|\mathbf{z})Pr(\mathbf{z})d\mathbf{z}. \tag{4}$$

No matter how complicated $Pr(\mathbf{x})$ is, it can be re-constructed using relatively simple $Pr(\mathbf{x}|\mathbf{z})$ and $Pr(\mathbf{z})$.

Example: Consider a 1D mixture of Gaussians. The latent variable $\mathbf{z}$ is discrete. The prior $Pr(\mathbf{z})$ is a categorical distribution with probability λ_n for each possible value of z. The conditional likelihood $Pr(x|z=n)$ of the data x given that the latent variable z takes value n is normally distributed with mean μ_n and variance σ_n^2:

$$Pr(z=n) = \lambda_n \quad (\sum_{n=1}^{N} \lambda_n = 1), \tag{5}$$

$$Pr(x|z) = \Phi(x; \mu_n, \sigma_n^2). \tag{6}$$

By (4), it is easy to observe:

$$Pr(x) = \sum_{n=1}^{N} Pr(x, z=n) \tag{7}$$

$$= \sum_{n=1}^{N} Pr(x|z=n) \cdot Pr(z=n) \tag{8}$$

$$= \sum_{n=1}^{N} \lambda_n \Phi(x; \mu_n, \sigma_n^2). \tag{9}$$

In a nonlinear and continuous setting where both the data $\mathbf{x}$ and the latent variable $\mathbf{z}$ are continuous and multivariate. We use a **standard multivariate Gaussian** as the prior of $Pr(\mathbf{z})$:

$$Pr(\mathbf{z}) = \Phi(\mathbf{z}; \mathbf{0}, \mathbf{I}). \tag{10}$$

We assume the conditional probability $Pr(\mathbf{x}|\mathbf{z}, \phi)$ is also Gaussian. Its mean is a nonlinear function $\mathbf{f}[\mathbf{z}, \phi]$ and its covariance is $\sigma^2\mathbf{I}$. This means that:

$$Pr(\mathbf{x}|\mathbf{z}, \phi) = \Phi\left(\mathbf{x}; \mathbf{f}[\mathbf{z}, \phi], \boldsymbol{\sigma^2}\mathbf{I}\right), \tag{11}$$

where $\mathbf{f}[\mathbf{z}, \phi]$ is describes by a DNN.

Again, by applying the marginalization trick, we have:

$$Pr(\mathbf{x}|\phi) = \int Pr(\mathbf{x}, \mathbf{z}|\phi) d\mathbf{z} \tag{12}$$

$$Pr(\mathbf{x}|\phi) = \int Pr(\mathbf{x}|\mathbf{z}, \phi) Pr(\mathbf{z}) d\mathbf{z} \tag{13}$$

$$Pr(\mathbf{x}|\phi) = \int \Phi\left(\mathbf{x}; \mathbf{f}[\mathbf{z}, \phi], \sigma^2 \mathbf{I}\right) \Phi\left(\mathbf{z}; \mathbf{0}, \mathbf{I}\right) \tag{14}$$

3 How to train the model?

In practice, we assume that the variance term σ^2 is known, and we aim to learn ϕ by MLL given the data $\{\mathbf{x}_i\}_{i=1}^{I}$:

$$\phi^* = \arg\max_{\phi} \sum_{i=1}^{I} \log\left[Pr(\mathbf{x}_i|\phi)\right], \tag{15}$$

which, however, is intractable!

We define a lower bound on the log-likelihood function. This is a function that is always less than or equal to the log-likelihood for a given value of ϕ (and it will also depend on some other parameters θ).

Jensen's Inequality: A concave function $g[\cdot]$ of the expectation of data y is greater than or equal to the expectation of the function of the data:

$$g[\mathbb{E}(y)] \geq \mathbb{E}[g(y)]. \tag{16}$$

When the concave function g is the logarithm, we have:

$$\log[\mathbb{E}(y)] \geq \mathbb{E}[\log(y)]. \tag{17}$$

Now we derive the lower bound for $\log[Pr(\mathbf{x}|\mathbf{\Phi})]$. The log-likelihood can be re-written as:

$$\log\left[Pr(\mathbf{x}|\phi)\right] = \log\left[\int Pr(\mathbf{x}, \mathbf{z}|\phi) d\mathbf{z}\right] \tag{18}$$

$$= \log\left[\int q(\mathbf{z}) \frac{Pr(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z})} d\mathbf{z}\right]. \tag{19}$$

Then according to the Jensen's inequality:

$$\log\left[\int q(\mathbf{z}) \frac{Pr(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z})} d\mathbf{z}\right] \geq \int q(\mathbf{z}) \log\left[\frac{Pr(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z})}\right] d\mathbf{z}. \tag{20}$$

The RHS is called the evidence lower bound or **ELBO**.

In practice, the function $q(\mathbf{z})$ is parameterized by θ, so the ELBO can be written as:

$$\text{ELBO}[\theta, \phi] = \int q(\mathbf{z}|\theta) \log \left[\frac{Pr(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} d\mathbf{z} \right]. \tag{21}$$

In order to solve (15), we just need to maximize $\text{ELBO}[\theta, \phi]$ as a function of θ and ϕ. We call a neural network architecture that computes such a ELBO the **Variational Auto-encoder**, or **VAE**.

4 The reformulation of ELBO

A useful and more interpretable way to write the ELBO is as follows:

$$\begin{aligned}
\text{ELBO}[\theta, \phi] &= \int q(\mathbf{z}|\theta) \log \left[\frac{Pr(\mathbf{x}|\mathbf{z}, \phi) Pr(\mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} d\mathbf{z} \right], && (22) \\
&= \int q(\mathbf{z}|\theta) \log \left[Pr(\mathbf{x}|\mathbf{z}, \phi) d\mathbf{z} \right] + \int q(\mathbf{z}|\theta) \log \left[\frac{Pr(\mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z}, && (23) \\
&= \underbrace{\int q(\mathbf{z}|\theta) \log \left[Pr(\mathbf{x}|\mathbf{z}, \phi) d\mathbf{z} \right]}_{\text{reconstruction loss}} - \underbrace{D_{KL} \left[q(\mathbf{z}|\theta) \| Pr(\mathbf{z}) \right]}_{\text{KL-divergence}}. && (24)
\end{aligned}$$

The two distribution in the KL-divergence part are Gaussian and therefore this term has closed-form expression. However, the reconstruction loss term is still intractable (but we know it is an expectation over $q(\mathbf{z}|\mathbf{x}, \theta)$). We can use *Monte-Carlo* estimate to approximate the reconstruction loss. Consider a very rough approximation, we can just sample one realization $\mathbf{z}^*$ from $q(\mathbf{z}|\mathbf{x}, \theta)$, which leads to:

$$ELBO[\theta, \phi] \simeq \log \left[Pr(\mathbf{x}|\mathbf{z}^*, \phi) \right] - D_{KL} \left[q(\mathbf{z}|\theta) \| Pr(\mathbf{z}) \right] \tag{25}$$

5 The reparameterization trick

For the neural network architecture we have just built above, there is a sampling step which is not differentiable. We use reparameterization to tackle this. Recall the encoder part output the mean and the covariance of $q(\mathbf{z}|\mathbf{x}, \theta)$, we than can turn to use the following relation to sample from $q(\mathbf{z}|\mathbf{x}, \theta)$:

$$\mathbf{z}^* = \mu + \mathbf{\Sigma}^{1/2} \cdot \epsilon, \tag{26}$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We then feed $\mathbf{z}^*$ to the decoder part.

6 Generation

Upon proper training, the posterior $q(\mathbf{z}|\mathbf{x}, \theta)$ should be very close to the prior $Pr(\mathbf{z})$ (this links to the tightness of the ELBO, you can find the proof if you are interested). We therefore are able to generate samples by simply generate from a standard Gaussian, and then feed the vector to the decoder part.